

CR030: Student Registration with ID auto Increment by New Student

1. Change Request Details

Field	Details
Change Request ID	CR030
Title	Student Registration with ID auto Increment by New Student
Priority	High
Difficulty	Low
Change Type	Frontend & Backend Enhancement
Affected Version	Student Management System v1.0 baseline
Dependency	Existing Student Management System, User Authentication and Student Management Modules

2. Business Requirement

Currently, student accounts are created through the administrator student management functionality. The system should be enhanced to allow a new student to register independently through a student self-registration facility.

When a student completes the registration process successfully, the system should automatically create the student's account and profile and generate a unique Student ID.

The Student ID must be automatically generated by the database and must never be manually entered or selected by the student. If an administrator deletes an existing student, the deleted Student ID must not be assigned to another student in the future.

For example, if the system has Student IDs 1, 2, 3, 4 and 5 and Student ID 3 is deleted, the next newly registered student should receive the next available sequential ID according to the database auto-increment sequence, rather than reusing ID 3.

The main purpose of this change is to simplify student registration, reduce administrator workload, and ensure that each student has a unique and permanently assigned Student ID.

3. Current Behaviour

- Student records are currently created through the administrator student management interface.
- The existing student form allows an administrator to create a new student account.
- Student registration information includes registration number, first name, last name, email, phone number and password.
- The backend creates a user account and then creates the corresponding student profile.
- The students table already uses `student_id INT AUTO_INCREMENT PRIMARY KEY`.
- Student IDs are therefore generated by the database when a student record is inserted.
- Existing Edit and Delete operations identify students using `student_id`.

- Student creation currently requires administrator authentication and the student creation route is restricted to the ADMIN role.
- Existing student creation is handled through POST /api/students/.
- Existing authentication and role-based authorization must remain protected.

The current student creation process requires an administrator to enter the student information instead of allowing the student to register independently.

4. Requested Change

Add a student self-registration facility to the Student Management System.

The implementation must:

- Provide a registration interface that can be accessed by a new student.
- Allow students to enter their required registration information.
- Create a new student user account after successful validation.
- Automatically generate a unique Student ID.
- Ensure the Student ID is generated by the database rather than entered manually.
- Ensure Student IDs are not duplicated.
- Ensure a deleted Student ID is not reused for a future student.
- Continue the Student ID sequence when new students register.
- Validate required registration fields.
- Prevent duplicate registration numbers.
- Prevent duplicate email addresses.
- Securely hash the student's password before storing it.
- Create the user account and student profile as a consistent transaction.
- Return the newly generated Student ID after successful registration.
- Allow the newly registered student to log in using the created credentials.
- Preserve existing administrator CRUD functionality.
- Preserve existing administrator authentication and authorization.
- Prevent unauthorized users from accessing administrator-only student management functions.

5. Scope

In Scope

- New student self-registration page/form.
- Student registration form validation.
- Automatic Student ID generation.
- Database-controlled Student ID sequence.
- Unique Student ID enforcement.
- Prevention of deleted Student ID reuse.
- Student account creation.
- Student profile creation.
- Duplicate registration number validation.
- Duplicate email validation.
- Password hashing.
- Registration success message.
- Display of newly generated Student ID.
- Student login after successful registration.
- Responsive registration interface.
- Preservation of existing administrator CRUD functionality.
- Preservation of existing authentication and authorization.

Out of Scope

- Changes to administrator authentication.
- Changes to administrator authorization roles.
- Changes to the existing Edit Student functionality.
- Changes to the existing Delete Student functionality except where required to preserve ID sequencing.
- Student ID manual selection.
- Manual Student ID editing.
- Reusing deleted Student IDs.
- Database redesign unrelated to student registration.
- Server-side student searching or filtering.
- Pagination.
- Changes to existing student profile features unrelated to registration.

6. Expected / Potentially Affected Files

The following files may be modified or added:

- frontend/pages/RegisterPage.jsx (*new*)
- frontend/components/StudentRegistrationForm.jsx (*new, optional*)
- frontend/services/api.js
- frontend/services/authService.js
- frontend/pages/LoginPage.jsx (*if a registration link is added*)
- frontend/pages/DashboardPage.jsx (*only if required to display the newly registered student*)
- frontend/styles/register.css (*new*)
- backend/controllers/studentController.js
- backend/models/studentModel.js
- backend/routes/studentRoutes.js
- backend/middleware/authMiddleware.js (*only if route protection requires modification*)
- backend/schema.sql (*only if database constraints or configuration require modification*)

Any additional files modified during implementation must be identified and justified in the Implementation Report.

The existing database schema already defines `student_id` as an `INT AUTO_INCREMENT PRIMARY KEY`, which provides the required database-generated unique identifier mechanism.

7. Functional & Security Requirements

Functional Requirements

FR-001: A student registration interface shall be available to new students.

FR-002: Students shall be able to enter the required registration information.

FR-003: The system shall automatically generate a Student ID after successful registration.

FR-004: Students shall not be allowed to manually enter or select the Student ID.

FR-005: Student IDs shall be generated using the database auto-increment mechanism.

FR-006: Each Student ID shall be unique.

FR-007: A deleted Student ID shall not be reused for a future student.

FR-008: New student registrations shall continue from the existing Student ID sequence.

FR-009: The system shall validate all required registration fields before creating the account.

FR-010: The system shall prevent duplicate registration numbers.

FR-011: The system shall prevent duplicate email addresses.

FR-012: The student's password shall be securely hashed before being stored in the database.

FR-013: The system shall create the user account and student profile successfully as one registration operation.

FR-014: The system shall return or display the newly generated Student ID after successful registration.

FR-015: The newly registered student shall be able to log in using the registered credentials.

FR-016: Existing administrator Edit and Delete functionality shall continue to operate correctly using the correct student_id.

FR-017: Existing administrator student management functionality shall remain available.

Security Requirements

SEC-001: Student registration shall not provide access to administrator privileges.

SEC-002: Students shall receive the STUDENT role when their account is created.

SEC-003: Passwords shall never be stored as plain text.

SEC-004: Existing JWT authentication and role-based authorization shall remain unchanged unless modification is necessary for the registration route.

SEC-005: Administrator-only operations shall remain protected by authentication and role authorization.

SEC-006: Student IDs shall not be accepted from the client as a trusted value.

SEC-007: The server/database shall determine the generated Student ID.

SEC-008: The registration process shall prevent duplicate accounts based on unique registration numbers and email addresses.

SEC-009: User input shall be validated and safely handled without unsafe HTML rendering.

The existing system already uses password hashing with bcrypt during student creation and assigns the created account the STUDENT role.

The existing student database also defines unique constraints for registration_no and email, which support duplicate prevention.

8. Acceptance Criteria

The change request shall be considered successfully implemented when:

AC-001: A new student can access the student registration form.

AC-002: A student can successfully submit valid registration information without administrator assistance.

AC-003: The system automatically generates a Student ID after successful registration.

AC-004: The student does not have to enter the Student ID manually.

AC-005: The generated Student ID is unique.

AC-006: If a student with an existing Student ID is deleted, that Student ID is not reused for a newly registered student.

AC-007: New students continue receiving new Student IDs according to the database auto-increment sequence.

AC-008: Duplicate registration numbers are rejected.

AC-009: Duplicate email addresses are rejected.

AC-010: The password is securely hashed before being stored.

AC-011: The newly registered student receives the STUDENT role and does not receive administrator privileges.

AC-012: The system displays a successful registration message containing the newly generated Student ID.

AC-013: The newly registered student can log in using the registered username/registration number and password.

AC-014: Existing administrator Edit functionality continues to update the correct student.

AC-015: Existing administrator Delete functionality continues to delete only the selected student.

AC-016: Existing administrator authentication and authorization continue to function correctly.

AC-017: Unauthorized users cannot access administrator-only student management operations.

AC-018: No duplicate Student IDs exist after multiple student registrations and deletions.

9. Expected Outcome

After implementation, new students will be able to register themselves through the Student Management System without requiring an administrator to manually create their account.

Once registration is completed successfully, the system will automatically create the student's user account and profile and assign a unique Student ID.

The Student ID will be generated by the database using the existing auto-increment mechanism. Student IDs will not be manually entered, duplicated, or reused after deletion.

For Example

Registration Event	Student ID
First student registers	1
Second student registers	2
Third student registers	3
Fourth student registers	4
Student ID 3 is deleted	3 becomes unavailable
New student registers	5
Next student registers	6

Therefore, deleting Student ID 3 will not cause the system to assign Student ID 3 to another student. The sequence continues with the next generated ID.

The existing administrator CRUD operations, authentication, authorization, database integrity and student management functionality will remain available.

10. Implementation Report Requirement

After implementation, the Final Exam Software Change Implementation Report shall include the following sections:

10.1 Implementation Summary

Describe how the student self-registration functionality was implemented and how automatic Student ID generation was introduced.

The report should explain that the Student ID is generated automatically by the database and is not supplied by the student through the frontend.

10.2 Affected Files

The report shall list all files that were created or modified during implementation.

For example:

- Registration page/component.
- Student authentication/registration service.
- Student controller.
- Student model.
- Student route.
- Database schema, if modified.
- Registration stylesheet.

Each additional file should include a short explanation of why it was changed.

10.3 Important Technical Changes

The report shall explain:

- Student self-registration workflow.
- Automatic Student ID generation.
- Database auto-increment mechanism.
- Unique Student ID handling.
- Prevention of deleted ID reuse.
- Duplicate registration number validation.
- Duplicate email validation.
- Password hashing.
- User and student profile creation.
- Registration success response.
- Student login after registration.

The existing students table uses `student_id INT AUTO_INCREMENT PRIMARY KEY`, so the implementation should rely on this database mechanism rather than generating IDs in React or JavaScript.

10.4 Security Changes

The report shall document:

- Password hashing.
- Student role assignment.
- Input validation.
- Duplicate account prevention.
- Authentication handling.
- Authorization protection.
- Prevention of client-controlled Student IDs.
- Protection of administrator-only functionality.

The existing student creation controller already performs password hashing and creates the account with the STUDENT role.

10.5 Test Cases and Results

The implementation report shall include test cases such as:

Test Case ID	Test Scenario	Expected Result	Status
TC-001	Open student registration page	Registration form is displayed	Pass/Fail
TC-002	Register with valid information	Student account is created successfully	Pass/Fail
TC-003	Register with duplicate registration number	Registration is rejected	Pass/Fail
TC-004	Register with duplicate email	Registration is rejected	Pass/Fail
TC-005	Register with missing required field	Validation error is displayed	Pass/Fail
TC-006	Check generated Student ID	Student ID is automatically generated	Pass/Fail
TC-007	Register multiple students	Each student receives a different Student ID	Pass/Fail
TC-008	Delete an existing student	Student is deleted successfully	Pass/Fail
TC-009	Register a new student after deletion	Deleted Student ID is not reused	Pass/Fail
TC-010	Login using newly registered account	Student can log in successfully	Pass/Fail
TC-011	Attempt administrator function as student	Access is denied	Pass/Fail
TC-012	Edit student as administrator	Correct student is updated	Pass/Fail
TC-013	Delete student as administrator	Correct student is deleted	Pass/Fail
TC-014	Check duplicate Student IDs	No duplicate Student IDs exist	Pass/Fail

10.6 Acceptance Criteria Verification

The final report shall verify each acceptance criterion from AC-001 to AC-018 and provide the corresponding test result or evidence.

Example:

Acceptance Criterion	Verification	Result
AC-001	Registration page tested	Pass
AC-002	Valid student registration tested	Pass
AC-003	Automatic ID generation tested	Pass
AC-004	Student ID field is not available in registration form	Pass
AC-005	Multiple registrations tested	Pass
AC-006	Deleted ID reuse tested	Pass
AC-007	Sequential ID generation tested	Pass
AC-008	Duplicate registration number tested	Pass
AC-009	Duplicate email tested	Pass

AC-010	Password storage verified as hashed	Pass
AC-011	Student role verified	Pass
AC-012	Generated Student ID displayed after registration	Pass
AC-013	Student login tested	Pass
AC-014	Administrator Edit tested	Pass
AC-015	Administrator Delete tested	Pass
AC-016	Authentication/authorization tested	Pass
AC-017	Unauthorized access tested	Pass
AC-018	Student ID uniqueness verified	Pass

10.7 Screenshots / Evidence

The implementation report shall include relevant screenshots such as:

1. Student Registration Page.
2. Student Registration Form with required fields.
3. Successful Student Registration message.
4. Automatically generated Student ID.
5. Student record displayed in the Administrator Dashboard.
6. Database showing the generated Student ID.
7. Student login page.
8. Successful Student Dashboard login.
9. Administrator Delete Student operation.
10. New student registration after deletion showing that the deleted Student ID was not reused.
11. Duplicate registration number validation.
12. Duplicate email validation.
13. Unauthorized access attempt by a student.

10.8 Final Status

Final Status: Successfully Implemented / Pending Testing

The final status shall be marked as **Successfully Implemented** only after all functional, security and acceptance tests have passed.

10.9 Outstanding Issues

Any remaining issues discovered during implementation or testing shall be documented here.

If all acceptance criteria and test cases pass, the report should state:

Outstanding Issues: None.

Final Change Summary

CR-030 introduces student self-registration and automatic Student ID generation.

Students will be able to register independently, while the system will automatically generate their Student ID using the database auto-increment mechanism.

The implementation must ensure that Student IDs remain unique and that a deleted Student ID is **never reused**. Existing administrator CRUD operations, authentication, authorization, password security and database integrity must continue to operate without unnecessary changes.